L10: SimPy Fundamentals

Simulation Without a Black Box

Outline

Learning Objectives

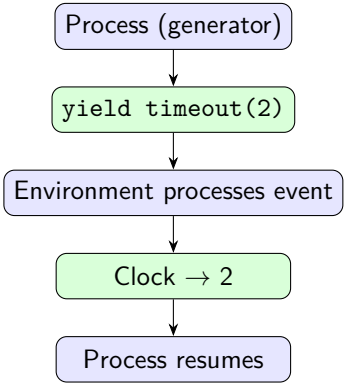
By the end of this lecture you will be able to:

- Describe SimPy's execution model: generators that yield events.
- Create an `Environment`, define processes, and call `env.run()`.
- Allocate and release a `Resource` using the `with` block pattern.
- Use `env.timeout()` to model service durations and other delays.
- Instrument a process to collect waiting-time and queue-length data.

SimPy Philosophy: Generators Yield Events

Three-sentence summary

1. A **process** is a Python generator function that yields SimPy event objects.
2. The **environment** maintains a priority queue of (time, event) pairs and advances the clock to each event in order.
3. When a process yields an event, it *suspends*; when the event fires, the environment *resumes* the process.



simpy.Environment and env.run()

```python
import simpy

def clock(env, interval):
    """Print the time every `interval` units."""
    while True:
        print(f"Tick at t={env.now:.1f}")
        yield env.timeout(interval)

env = simpy.Environment()
env.process(clock(env, interval=1.5))
env.run(until=5)
# Tick at t=0.0
# Tick at t=1.5
# Tick at t=3.0
# Tick at t=4.5
```

- `env.process(gen)` registers a generator as a SimPy process.
- `env.run(until=T)` advances simulation time to T then stops.
- `env.now` returns the current simulation clock value.

simpy.Resource: Request / Release

```
server = simpy.Resource(env, capacity=1)

def customer(env, name, server,
             svc_time, waits):
    arrive = env.now
    with server.request() as req:
        yield req               # wait for
        ↪  server
        waits.append(
            env.now - arrive)   # record wait
        yield env.timeout(
            svc_time)           # in service
    # server released automatically
```

The with idiom

- `with server.request() as req:` creates a request event.
- `yield req` suspends the process until the server is free.
- The `with` block releases the server when the indented code completes — even if an exception occurs.
- Never call `req.release()` manually when using `with`.

env.timeout: Modelling Service Durations

```
import numpy as np

def barista(env, rng, server):
    """Serve customers. Service time ~ Exp(mu=2 cups/min)."""
    mu = 2.0
    while True:
        with server.request() as req:
            yield req
            brew_time = rng.exponential(scale=1/mu)
            yield env.timeout(brew_time)    # service happens here
```

- `env.timeout(delay)` creates an event that fires `delay` time units later.
- The process is suspended during the timeout — other processes can run.
- **Key**: draw random variates *just before* `yield env.timeout()` so the sample is consumed when service begins, not when the customer arrives.

Coffee Shop Walkthrough (Complete Example)

```
import simpy, numpy as np

def customer(env, server, rng, waits):
    arrive = env.now
    with server.request() as req:
        yield req
        waits.append(env.now - arrive)          # record wait
        yield env.timeout(rng.exponential(0.5)) # service

def arrivals(env, server, rng, waits):
    while True:
        yield env.timeout(rng.exponential(1/1.5))
        env.process(customer(env, server, rng, waits))

rng    = np.random.default_rng(42)
env    = simpy.Environment()
server = simpy.Resource(env, capacity=1)
waits  = []
env.process(arrivals(env, server, rng, waits))
env.run(until=1000)
print(f"Mean wait: {np.mean(waits):.3f}  (theory: {1.5/(2*(2-1.5)):.3f})")
```

Monitoring: Collecting Metrics Inside the Generator

```
queue_log = []   # (time, queue_length)

def customer(env, server, rng, waits):
    arrive = env.now
    queue_log.append(
        (arrive, len(server.queue)))
    with server.request() as req:
        yield req
        waits.append(env.now - arrive)
        yield env.timeout(
            rng.exponential(1))
```

What to monitor

- **Arrival time**: `env.now` before `yield req`
- **Wait time**: difference after `yield req`
- **Queue length**: `len(server.queue)` at arrival
- **Utilisation**: `server.count / server.capacity`

Collect event-driven samples; post-process with pandas or NumPy outside the simulation loop.

env.now: Timestamps and Event Logging

```
events = []  # structured event log

def customer(env, cid, server, rng):
    events.append(dict(t=env.now, cid=cid, event="arrive"))
    with server.request() as req:
        yield req
        events.append(dict(t=env.now, cid=cid, event="start_svc"))
        yield env.timeout(rng.exponential(1))
        events.append(dict(t=env.now, cid=cid, event="depart"))

# Post-simulation analysis
import pandas as pd
df = pd.DataFrame(events)
waits = (df[df.event=="start_svc"].t.values
         - df[df.event=="arrive"].t.values)
```

Structured event logs are the most flexible monitoring approach: they let you compute *any* derived metric after the run without re-running.

Key Takeaways

Core ideas from L10

1. SimPy's execution model is purely **generator-based**: no threads, no callbacks.
2. `env.process()` registers a generator; `env.run()` drives the clock.
3. The `with server.request() as req:` idiom safely requests and auto-releases a resource.
4. `env.timeout(t)` is how simulated time passes — it is the only mechanism.
5. Collect metrics **inside** the generator at event points; post-process outside.
6. `env.now` gives the current clock value anywhere in a process.

Next: L11 — The M/M/1 queue: theory, simulation, and the hockey-stick effect.